Mobile Demo Script Explanation -- Loadme (Reversing, Challenge 05)

Olawoyin Joseph Pamilerin, ISU520196

In one sentence: Each loading stage XOR-decrypts the next stage using a key derived from data already available on-device (the package name).

Background -- what the terms below mean

What does "Reversing" mean, as a category? A **Reversing** challenge gives you someone else's already-built app (an APK) and asks you to figure out, by reading its code, some secret value it's hiding -- a PIN, a password, a flag string.

What is an APK, and what does "decompile" mean? An APK is the installable file for an Android app. It ships as compiled code that isn't directly readable, so we **decompile** it -- turn it back into readable, Java-like source code -- using a free tool called **jadx** (opened as **jadx-gui**).

Why is there no exploit app to build here? Reversing challenges usually aren't solved by writing an Android app at all -- they're solved by reading the decompiled logic and then writing a small script (usually Python) that re-implements or defeats that same logic on a regular computer, offline.

How the APK was obtained

The target APK was **downloaded directly from the challenge's own page** on challs.reyammer.io, before doing anything else. A copy of it is kept at:

```
C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Reversing/05 Loadme\Original Target APK.apk
```

How the solution was built

No Android app was built for this one at all. It was solved by **decompiling the target with jadx** and then writing a small **Python script** (or, in a couple of cases, working the logic out by hand) that re-implements or defeats the exact same check on a regular computer, offline. The recovered value was then typed directly into the challenge's own flag box.

The vulnerability / technique, in plain terms

Reverse-engineered a 3-stage dynamically-loaded APK: stage 1 XOR-decrypted with the package name, stage 2 XOR-decrypted with a fixed string key, final stage contained the hardcoded flag.

Files to open

1. Open the target APK in jadx-gui:

```
C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Reversing/05 Loadme\Original Target APK.apk
```

2. DoStuff.java (jadx decompiled):

```
C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Reversing/05 Loadme\Analysis and Scripts\Challenge 12 Lo
```

In jadx-gui's left tree: Source code -> com -> mobisec -> dexclassloader -> DoStuff.java

3. decode.py:

C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Reversing/05 Loadme\Analysis and Scripts\Challenge 12 Lo

What to say -- explanation of the exploit, step by step

1. I decompiled the APK and found it dynamically loads a 3-stage payload.

2. I identified that stage 1 is XOR-decrypted using the app's own package name as the key, and stage 2 using a fixed string key.

3. I wrote a script to reproduce both decryption steps and extracted the final stage, which contained the hardcoded flag.

If an emulator isn't available

If asked how you got/confirmed the flag: "I decompiled the target with jadx, recovered the secret value with a script (no exploit app was built or uploaded for this one), and typed the recovered flag directly into the challenge site's own flag box." There is no APK submission or platform analysis log for this challenge -- don't claim one.

Result

The flag this challenge produced:

MOBISEC{dynamic_code_loading_can_make_everything_tricky_eh?}

Source: Typed into the CTFd flag box; solve confirmed live from /user